

National University of Computer and Emerging Sciences, Lahore Campus



Course:	Design and Analysis of Algorithms	Course Code:	CS-2009
Program:	BS (CS, DS)	Semester:	Spring 2025
Due Date:	October 31, 2025	Total Marks:	100
Sections:	CS-5(C,D), DS-5(B,C)	Exam:	A#2

Instructions

1. Write clean code with functions and comments where necessary.
2. Plagiarism in any form will not be tolerated.
3. You need to create a separate code file for each question, naming it 23L-XXXX_A2_qX.cpp. Submissions that do not follow this naming convention will not be accepted.

Part 1: Divide & Conquer

Input/Output: Standard input to standard output. No interactive input. For example, use `freopen("input.txt", "r", stdin)`, `freopen("output.txt", "r", stdout)`.

Question 1 - Hidden Treasure Map (Largest All-1 Square)

You are given an $n \times n$ grid of 0/1. Find the **side length** of the largest square submatrix that contains only 1.

Input

n

```
row1
...
rown
```

Output

One integer: the **maximum k** such that there exists a $k \times k$ all-1 submatrix.

Requirements - Use **divide & conquer** over quadrants. Consider squares that **cross** the split.

- You may precompute prefix sums if you want, but the core must be recursive D&C (not DP-only).

- Time target: **$O(n^2 \log n)$** or better. Memory: **$O(n^2)$** .

Suggested functions

```
int largestSquare(const vector<vector<int>>& a);
bool allOnes(const vector<vector<int>>& pref, int r1,int c1,int r2,int c2); // if using
prefix sums
```

Question 2 - DNA Sequence Breaker

Given a string over {A, T, C, G}, find the **longest substring** where $\#A = \#T$ and $\#C = \#G$.

Input

```
n
S
n = |S| ( $1 \leq n \leq 2e5$ ). S has only A,T,C,G.
```

Output

One integer: the length of the longest valid substring.

Requirements - Use **divide & conquer** on indices: solve left, right, and **crossing** substrings.

- For crossing, match balance pairs from left suffixes to right prefixes.

- Time target: **$O(n \log n)$** with hash maps / unordered_map. Memory: **$O(n)$** .

Hint for balances Let $b1 = \#A - \#T$, $b2 = \#C - \#G$. A substring is valid iff the prefix at its right end and the prefix at its left-1 have **equal** ($b1$, $b2$).

Think why this would be true, can you formulate a proof?

Suggested functions

```
int longestBalanced(const string& s);
```

Question 3 (Bonus)

Given in a separate file.

Part 2: Dynamic Programming

Answer all questions clearly. For each problem, you must:

- Write the recursive relation or recurrence first.
- Explain your DP state design and transitions.
- Then provide your final code implementation.

Question 4 - Holiday Planner

You are planning a vacation for D days. Each day you can choose one of three activities: A[i], B[i], or C[i], each giving a certain happiness value. You cannot choose the same activity on two consecutive days. Find the maximum total happiness you can achieve.

Input

```
D
A1 B1 C1
A2 B2 C2
...
AD BD CD
```

Example Input:

```
3
10 40 70
20 50 80
```

```
30 60 90
```

Output:

```
210
```

Tasks

1. Define a recursive function $f(\text{day}, \text{last})$ returning the maximum happiness from day to D.
2. Derive and write the recurrence.
3. Implement a bottom-up DP solution.
4. Show a small example table for 3 days.

Question 5 - The Bag Problem

You have N items, each with a weight $w[i]$ and value $v[i]$. You can carry items in a bag with capacity W . Find the maximum total value possible without exceeding the capacity.

Input

```
N W
w1 v1
w2 v2
...
wN vN
```

Example Input:

```
4 5
2 3
1 2
3 4
2 2
```

Output:

```
7
```

Tasks

1. Write a recursive definition $f(i, \text{remainingWeight})$ – the best value you can get considering items $i \dots N$.
2. Explain the recurrence and base cases.
3. Convert to a bottom-up DP.
4. Show a partial DP table for small input.

Question 6 - Counting Numbers without Forbidden Digits

You are given two integers L and R ($1 \leq L \leq R \leq 10^{18}$). Count how many numbers between L and R (inclusive) do not contain the digits 4 or 9.

Input

L R

Output

Count of valid numbers

Example Input:

1 20

Output:

18

Tasks

1. Define a recursive function $\text{count}(\text{pos}, \text{tight}, \text{hasBad})$ exploring digits from left to right.
2. Write the recurrence and meaning of each state.
3. Memoize or convert to bottom-up DP.
4. Provide code that handles values up to 10^{18} .
5. Manually show working for all numbers ≤ 50 .

Question 7 (Bonus)

You are given three integers L , R , and K ($1 \leq L \leq R \leq 10^{18}$, $1 \leq K \leq 100$). Count how many numbers between L and R (inclusive) have a digit sum divisible by K .

Input

L R K

Output

Count of valid numbers

Example Input:

1 50 5

Output:

9

Tasks

1. Define a recursive function $f(\text{pos}, \text{sumMod}, \text{tight})$ that counts valid numbers from position pos with current sum mod K .
 2. Derive the recurrence clearly and describe transitions.
 3. Explain how memoization avoids recomputation.
 4. Implement a digit DP solution that works efficiently up to 10^{18} .
 5. Show hand-worked reasoning for numbers ≤ 100 when $K = 5$.
-